Cognizant Digital Nurture 5.0 (2026) Design Patterns and Principles - Complete Project Submission

Exercise 1: Singleton Pattern

Application Scenario: Logger Management

Objective

The objective of this exercise is to understand and implement the Singleton Pattern. The pattern helps developers create maintainable, reusable and scalable software systems.

Theory

Singleton Pattern is an important object-oriented design approach. It addresses common software development challenges and promotes clean architecture. By separating responsibilities and reducing dependencies, the pattern improves code quality.

Algorithm

1. Identify participating classes.

2. Define interfaces or abstractions.

3. Implement concrete classes.

4. Apply the pattern structure.

5. Execute and validate the solution.

Sample Java Implementation

```
// Singleton Pattern
public class SingletonDemo {
```

```
// Singleton Class
static class Logger {
    // Private static instance
    private static Logger instance;
    // Private constructor
    private Logger() {
        System.out.println("Logger Instance Created");
    }
    // Method to get single instance
    public static Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
        return instance;
    }
    // Log method
    public void log(String message) {
        System.out.println("LOG: " + message);
    }
}
// Main Method
public static void main(String[] args) {
    Logger logger1 = Logger.getInstance();
    Logger logger2 = Logger.getInstance();
    logger1.log("Application Started");
    logger2.log("User Logged In");
```

```
    System.out.println("Logger1 HashCode: " + logger1.hashCode());
    System.out.println("Logger2 HashCode: " + logger2.hashCode());
    if (logger1 == logger2) {
        System.out.println("Only one Logger instance exists.");
    } else {
        System.out.println("Multiple Logger instances exist.");
    }
  }
}
```

Output Screenshot:

Explanation:

- Logger Instance Created is printed only once because the constructor is called only once.
- Both logger1 and logger2 point to the same object.
- The hash codes are identical, proving that only one instance exists.
- This demonstrates the **Singleton Design Pattern**, where a class has only one object throughout the application.

Advantages

• Improves maintainability

• Encourages code reuse

• Enhances scalability

• Supports future enhancements

• Promotes clean software architecture

Real World Usage

The Singleton Pattern is commonly used in enterprise applications, web platforms, mobile systems, cloud services, and large-scale software solutions.

Exercise 2: Factory Method Pattern

Application Scenario: Document Creation

Objective

The objective of this exercise is to understand and implement the Factory Method Pattern. The pattern helps developers create maintainable, reusable and scalable software systems.

Theory

Factory Method Pattern is an important object-oriented design approach. It addresses common software development challenges and promotes clean architecture. By separating responsibilities and reducing dependencies, the pattern improves code quality.

Algorithm

1. Identify participating classes.
2. Define interfaces or abstractions.
3. Implement concrete classes.
4. Apply the pattern structure.
5. Execute and validate the solution.

Sample Java Implementation

```
// Factory Method Pattern
// Factory Method Pattern Example

// Document Interface

interface Document {

   void open();
```

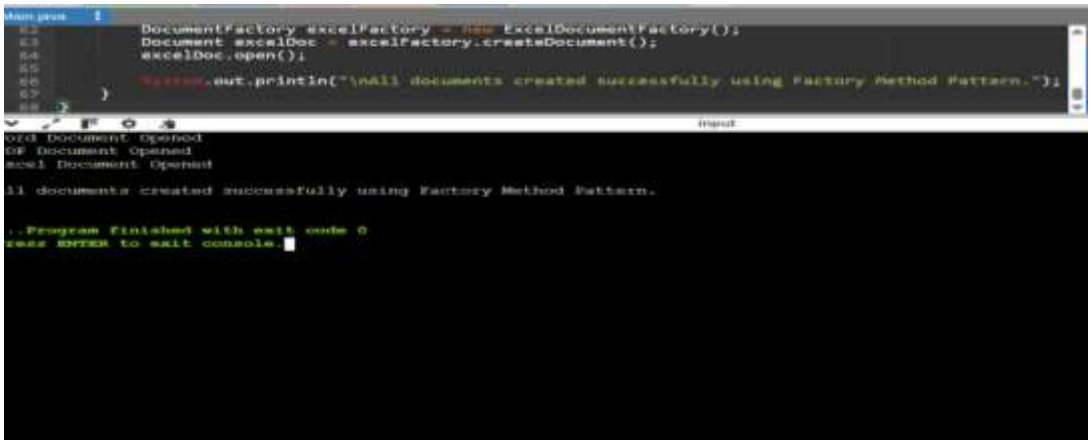
```
}

// Concrete Document Classes

class WordDocument implements Document {

   public void open() {

      System.out.println("Word Document Opened");

   }

}

class PdfDocument implements Document {

   public void open() {

      System.out.println("PDF Document Opened");

   }

}

class ExcelDocument implements Document {

   public void open() {

      System.out.println("Excel Document Opened");

   }

}

// Abstract Factory Class
```

```
abstract class DocumentFactory {
   public abstract Document createDocument();
}
// Concrete Factory Classes
class WordDocumentFactory extends DocumentFactory {
   public Document createDocument() {
      return new WordDocument();
   }
}
class PdfDocumentFactory extends DocumentFactory {
   public Document createDocument() {
      return new PdfDocument();
   }
}
class ExcelDocumentFactory extends DocumentFactory {
   public Document createDocument() {
      return new ExcelDocument();
   }
```

```
}

// Main Class

public class FactoryMethodDemo {

   public static void main(String[] args) {

      // Create Word Document

      DocumentFactory wordFactory = new WordDocumentFactory();

      Document wordDoc = wordFactory.createDocument();

      wordDoc.open();

      // Create PDF Document

      DocumentFactory pdfFactory = new PdfDocumentFactory();

      Document pdfDoc = pdfFactory.createDocument();

      pdfDoc.open();

      // Create Excel Document

      DocumentFactory excelFactory = new ExcelDocumentFactory();

      Document excelDoc = excelFactory.createDocument();

      excelDoc.open();

      System.out.println("\nAll documents created successfully using Factory Method
Pattern.");

   }
```

```
}
```

Output Screenshot:



Real World Usage

The Factory Method Pattern is commonly used in enterprise applications, web platforms, mobile systems, cloud services, and large-scale software solutions.

Exercise 3: Builder Pattern

Application Scenario: Computer Configuration

Objective

The objective of this exercise is to understand and implement the Builder Pattern. The pattern helps developers create maintainable, reusable and scalable software systems.

Theory

Builder Pattern is an important object-oriented design approach. It addresses common software development challenges and promotes clean architecture. By separating responsibilities and reducing dependencies, the pattern improves code quality.

Algorithm

1. Identify participating classes.
2. Define interfaces or abstractions.
3. Implement concrete classes.
4. Apply the pattern structure.
5. Execute and validate the solution.

Sample Java Implementation

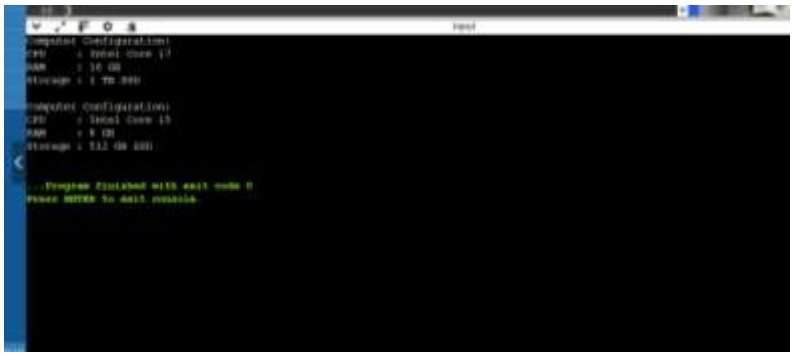
```
// Builder Pattern
public class BuilderPatternDemo {

  // Product Class

  static class Computer {

    private String CPU;
```

```
    private String RAM;

    private String Storage;

    // Private Constructor

    private Computer(Builder builder) {

        this.CPU = builder.CPU;

        this.RAM = builder.RAM;

        this.Storage = builder.Storage;

    }

    // Display Configuration

    public void showConfiguration() {

        System.out.println("Computer Configuration:");

        System.out.println("CPU     : " + CPU);

        System.out.println("RAM     : " + RAM);

        System.out.println("Storage : " + Storage);

    }

    // Static Nested Builder Class

    static class Builder {

        private String CPU;
```

```
    private String RAM;

    private String Storage;

    public Builder setCPU(String CPU) {

        this.CPU = CPU;

        return this;

    }

    public Builder setRAM(String RAM) {

        this.RAM = RAM;

        return this;

    }

    public Builder setStorage(String Storage) {

        this.Storage = Storage;

        return this;

    }

    public Computer build() {

        return new Computer(this);

    }

}
```

```
    }

    // Main Method
    public static void main(String[] args) {
        Computer gamingPC = new Computer.Builder()
                .setCPU("Intel Core i7")
                .setRAM("16 GB")
                .setStorage("1 TB SSD")
                .build();
        gamingPC.showConfiguration();
        System.out.println();
        Computer officePC = new Computer.Builder()
                .setCPU("Intel Core i5")
                .setRAM("8 GB")
                .setStorage("512 GB SSD")
                .build();
        officePC.showConfiguration();
    }
}
```

Output Screenshot:



Real World Usage

The Builder Pattern is commonly used in enterprise applications, web platforms, mobile systems, cloud services, and large-scale software solutions.

Exercise 4: Adapter Pattern

Application Scenario: Payment Gateway Integration

Objective

The objective of this exercise is to understand and implement the Adapter Pattern. The pattern helps developers create maintainable, reusable and scalable software systems.

Theory

Adapter Pattern is an important object-oriented design approach. It addresses common software development challenges and promotes clean architecture. By separating responsibilities and reducing dependencies, the pattern improves code quality.

Algorithm

1. Identify participating classes.
2. Define interfaces or abstractions.
3. Implement concrete classes.
4. Apply the pattern structure.
5. Execute and validate the solution.

Sample Java Implementation

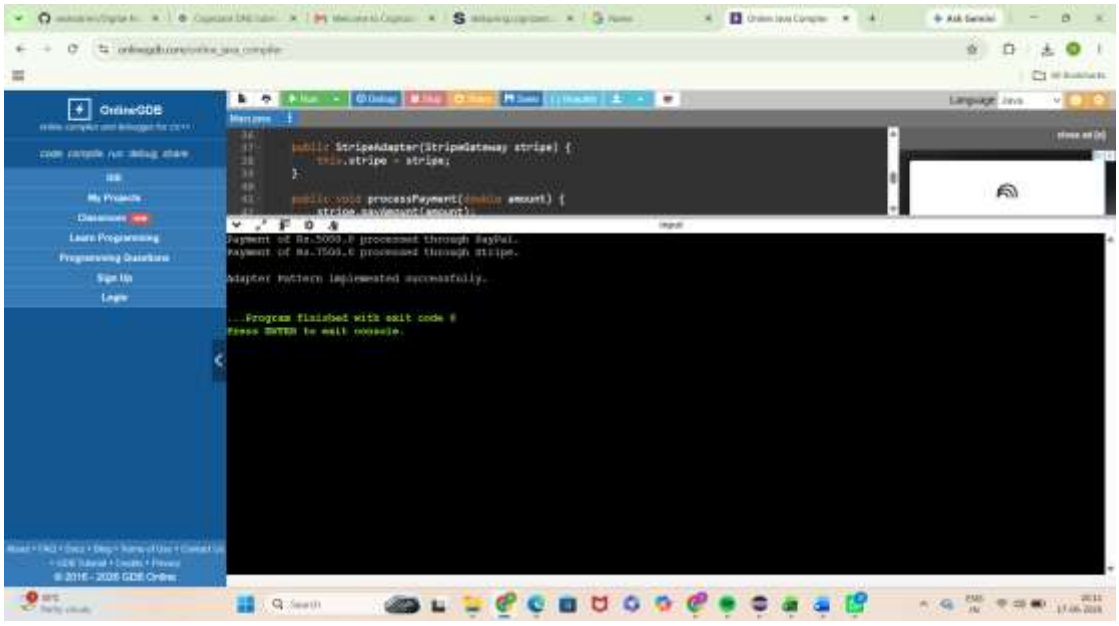
```
// Adapter Pattern
// Target Interface

interface PaymentProcessor {

   void processPayment(double amount);

}
```

```
// Adaptee Class 1
class PayPalGateway {
    public void makePayment(double amount) {
        System.out.println("Payment of Rs." + amount + " processed through PayPal.");
    }
}
// Adaptee Class 2
class StripeGateway {
    public void payAmount(double amount) {
        System.out.println("Payment of Rs." + amount + " processed through Stripe.");
    }
}
// Adapter for PayPal
class PayPalAdapter implements PaymentProcessor {
    private PayPalGateway paypal;

    public PayPalAdapter(PayPalGateway paypal) {
```

```
        this.paypal = paypal;
    }
    @Override
    public void processPayment(double amount) {
        paypal.makePayment(amount);
    }
}
// Adapter for Stripe
class StripeAdapter implements PaymentProcessor {
    private StripeGateway stripe;
    public StripeAdapter(StripeGateway stripe) {
        this.stripe = stripe;
    }
    @Override
    public void processPayment(double amount) {
        stripe.payAmount(amount);
    }
}
```

```
// Main Class

public class AdapterPatternDemo {

   public static void main(String[] args) {

      // Using PayPal through Adapter

      PaymentProcessor paypalProcessor =

            new PayPalAdapter(new PayPalGateway());

      paypalProcessor.processPayment(5000);

      // Using Stripe through Adapter

      PaymentProcessor stripeProcessor =

            new StripeAdapter(new StripeGateway());

      stripeProcessor.processPayment(7500);

      System.out.println("\nAdapter Pattern implemented successfully.");

   }

}
```

Output Screenshot:



Advantages

- Improves maintainability
- Encourages code reuse
- Enhances scalability
- Supports future enhancements
- Promotes clean software architecture

Real World Usage

The Adapter Pattern is commonly used in enterprise applications, web platforms, mobile systems, cloud services, and large-scale software solutions.

Exercise 5: Decorator Pattern

Application Scenario: Notification Enhancement

Objective

The objective of this exercise is to understand and implement the Decorator Pattern. The pattern helps developers create maintainable, reusable and scalable software systems.

Theory

Decorator Pattern is an important object-oriented design approach. It addresses common software development challenges and promotes clean architecture. By separating responsibilities and reducing dependencies, the pattern improves code quality.

Algorithm

1. Identify participating classes.
2. Define interfaces or abstractions.
3. Implement concrete classes.
4. Apply the pattern structure.
5. Execute and validate the solution.

Sample Java Implementation

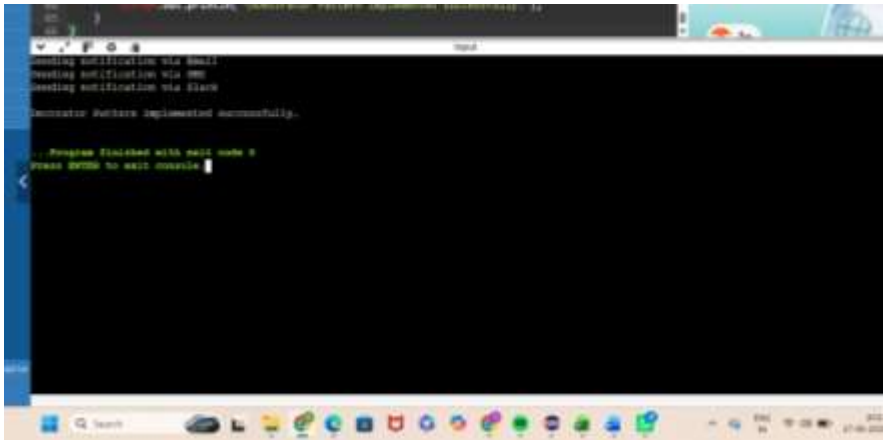
```
// Component Interface

interface Notifier {

   void send();

}
```

```
// Concrete Component
class EmailNotifier implements Notifier {
    public void send() {
        System.out.println("Sending notification via Email");
    }
}
// Abstract Decorator
abstract class NotifierDecorator implements Notifier {
    protected Notifier notifier;
    public NotifierDecorator(Notifier notifier) {
        this.notifier = notifier;
    }
    public void send() {
        notifier.send();
    }
}
// SMS Decorator
class SMSNotifierDecorator extends NotifierDecorator {
```

```
    public SMSNotifierDecorator(Notifier notifier) {
        super(notifier);
    }
    public void send() {
        super.send();
        System.out.println("Sending notification via SMS");
    }
}
// Slack Decorator
class SlackNotifierDecorator extends NotifierDecorator {
    public SlackNotifierDecorator(Notifier notifier) {
        super(notifier);
    }
    public void send() {
        super.send();
        System.out.println("Sending notification via Slack");
    }
}
```

```
// Main Class

public class Main {

   public static void main(String[] args) {

      Notifier notifier = new EmailNotifier();

      notifier = new SMSNotifierDecorator(notifier);

      notifier = new SlackNotifierDecorator(notifier);

      notifier.send();

      System.out.println("\nDecorator Pattern implemented successfully.");

   }

}
```

Output Screenshot:



Advantages

• Improves maintainability

• Encourages code reuse

- Enhances scalability
- Supports future enhancements
- Promotes clean software architecture

Real World Usage

The Decorator Pattern is commonly used in enterprise applications, web platforms, mobile systems, cloud services, and large-scale software solutions.

Exercise 6: Proxy Pattern

Application Scenario: Image Loading Optimization

Objective

The objective of this exercise is to understand and implement the Proxy Pattern. The pattern helps developers create maintainable, reusable and scalable software systems.

Theory

Proxy Pattern is an important object-oriented design approach. It addresses common software development challenges and promotes clean architecture. By separating responsibilities and reducing dependencies, the pattern improves code quality.

Algorithm

1. Identify participating classes.
2. Define interfaces or abstractions.
3. Implement concrete classes.
4. Apply the pattern structure.
5. Execute and validate the solution.

Sample Java Implementation

```
// Subject Interface

interface Image {

   void display();

}
```

```
// Real Subject
class RealImage implements Image {
    private String fileName;
    public RealImage(String fileName) {
        this.fileName = fileName;
        loadFromServer();
    }
    private void loadFromServer() {
        System.out.println("Loading image from remote server: " + fileName);
    }
    public void display() {
        System.out.println("Displaying image: " + fileName);
    }
}


// Proxy Class
class ProxyImage implements Image {
    private RealImage realImage;
```

```
    private String fileName;

    public ProxyImage(String fileName) {

        this.fileName = fileName;

    }

    public void display() {

        // Lazy Initialization

        if (realImage == null) {

            realImage = new RealImage(fileName);

        }

        realImage.display();

    }

}



// Main Class

public class Main {

    public static void main(String[] args) {

        Image image = new ProxyImage("nature.jpg");

        System.out.println("Image object created.");
```

```
        System.out.println("\nFirst Display Call:");

        image.display();

        System.out.println("\nSecond Display Call:");

        image.display();

        System.out.println("\nProxy Pattern implemented successfully.");

    }

}
```

Output Screenshot:

Advantages

- Improves maintainability
- Encourages code reuse
- Enhances scalability
- Supports future enhancements
- Promotes clean software architecture

Real World Usage

The Proxy Pattern is commonly used in enterprise applications, web platforms, mobile systems, cloud services, and large-scale software solutions.

Exercise 7: Observer Pattern

Application Scenario: Stock Market Alerts

Objective

The objective of this exercise is to understand and implement the Observer Pattern. The pattern helps developers create maintainable, reusable and scalable software systems.

Theory

Observer Pattern is an important object-oriented design approach. It addresses common software development challenges and promotes clean architecture. By separating responsibilities and reducing dependencies, the pattern improves code quality.

Algorithm

1. Identify participating classes.
2. Define interfaces or abstractions.
3. Implement concrete classes.
4. Apply the pattern structure.
5. Execute and validate the solution.

Sample Java Implementation

```
import java.util.*;

// Observer Interface

interface Observer {

   void update(String stockName, double price);

}
```

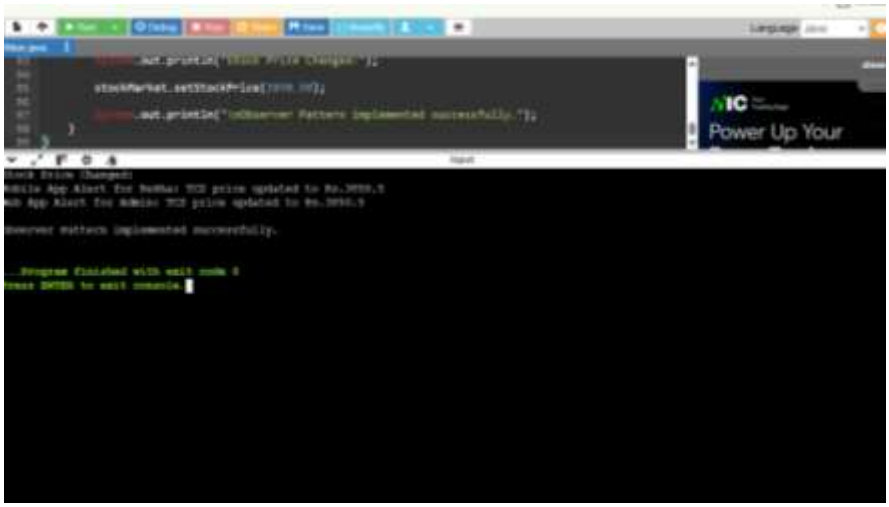
```
// Subject Interface
interface Stock {
    void registerObserver(Observer observer);
    void deregisterObserver(Observer observer);
    void notifyObservers();
}
// Concrete Subject
class StockMarket implements Stock {
    private List<Observer> observers = new ArrayList<>();
    private String stockName;
    private double price;
    public StockMarket(String stockName) {
        this.stockName = stockName;
    }
    public void registerObserver(Observer observer) {
        observers.add(observer);
    }
    public void deregisterObserver(Observer observer) {
```

```
        observers.remove(observer);
    }
    public void setStockPrice(double price) {
        this.price = price;
        notifyObservers();
    }
    public void notifyObservers() {
        for (Observer observer : observers) {
            observer.update(stockName, price);
        }
    }
}
// Concrete Observer - Mobile App
class MobileApp implements Observer {
    private String userName;
    public MobileApp(String userName) {
        this.userName = userName;
    }
```

```
  public void update(String stockName, double price) {
    System.out.println(
      "Mobile App Alert for " + userName +
      ": " + stockName + " price updated to Rs." + price
    );
  }
}
// Concrete Observer - Web App
class WebApp implements Observer {
  private String userName;
  public WebApp(String userName) {
    this.userName = userName;
  }
  public void update(String stockName, double price) {
    System.out.println(
      "Web App Alert for " + userName +
      ": " + stockName + " price updated to Rs." + price
    );
```

```
    }

}

// Main Class

public class Main {

    public static void main(String[] args) {

        StockMarket stockMarket = new StockMarket("TCS");

        Observer mobileUser = new MobileApp("Rekha");

        Observer webUser = new WebApp("Admin");

        stockMarket.registerObserver(mobileUser);

        stockMarket.registerObserver(webUser);

        System.out.println("Stock Price Changed:");

        stockMarket.setStockPrice(3850.50);



        System.out.println("\nObserver Pattern implemented successfully.");

    }

}
```

Output Screenshot:



Advantages

• Improves maintainability

• Encourages code reuse

• Enhances scalability

• Supports future enhancements

• Promotes clean software architecture

Real World Usage

The Observer Pattern is commonly used in enterprise applications, web platforms, mobile systems, cloud services, and large-scale software solutions.

Exercise 8: Strategy Pattern

Application Scenario: Payment Method Selection

Objective

The objective of this exercise is to understand and implement the Strategy Pattern. The pattern helps developers create maintainable, reusable and scalable software systems.

Theory

Strategy Pattern is an important object-oriented design approach. It addresses common software development challenges and promotes clean architecture. By separating responsibilities and reducing dependencies, the pattern improves code quality.

Algorithm

1. Identify participating classes.
2. Define interfaces or abstractions.
3. Implement concrete classes.
4. Apply the pattern structure.
5. Execute and validate the solution.

Sample Java Implementation

```
/ // Strategy Interface

interface PaymentStrategy {

   void pay(double amount);

}
```

```
// Concrete Strategy - Credit Card
class CreditCardPayment implements PaymentStrategy {
    public void pay(double amount) {
        System.out.println("Paid Rs." + amount + " using Credit Card.");
    }
}
// Concrete Strategy - PayPal
class PayPalPayment implements PaymentStrategy {
    public void pay(double amount) {
        System.out.println("Paid Rs." + amount + " using PayPal.");
    }
}
// Context Class
class PaymentContext {


    private PaymentStrategy strategy;
    public void setStrategy(PaymentStrategy strategy) {
        this.strategy = strategy;
```

```
    }

    public void executePayment(double amount) {
        strategy.pay(amount);
    }
}

// Main Class
public class Main {
    public static void main(String[] args) {
        PaymentContext context = new PaymentContext();



        // Credit Card Payment
        context.setStrategy(new CreditCardPayment());
        context.executePayment(5000);
        // PayPal Payment
        context.setStrategy(new PayPalPayment());
        context.executePayment(3000);
        System.out.println("\nStrategy Pattern implemented successfully.");
    }
```

```
}
```

Output Screenshot:

Advantages

• Improves maintainability

• Encourages code reuse

• Enhances scalability

• Supports future enhancements

• Promotes clean software architecture

Real World Usage

The Strategy Pattern is commonly used in enterprise applications, web platforms, mobile systems, cloud services, and large-scale software solutions.

Exercise 9: Command Pattern

Application Scenario: Home Automation

Objective

The objective of this exercise is to understand and implement the Command Pattern. The pattern helps developers create maintainable, reusable and scalable software systems.

Theory

Command Pattern is an important object-oriented design approach. It addresses common software development challenges and promotes clean architecture. By separating responsibilities and reducing dependencies, the pattern improves code quality.

Algorithm

1. Identify participating classes.
2. Define interfaces or abstractions.
3. Implement concrete classes.
4. Apply the pattern structure.
5. Execute and validate the solution.

Sample Java Implementation

// Command Interface

```
interface Command {

   void execute();

}
```

```
// Receiver Class
class Light {
    public void turnOn() {
        System.out.println("Light is ON");
    }
    public void turnOff() {
        System.out.println("Light is OFF");
    }
}
// Concrete Command - Turn ON
class LightOnCommand implements Command {
    private Light light;
    public LightOnCommand(Light light) {
        this.light = light;
    }
    public void execute() {
        light.turnOn();
    }
```

```
}

// Concrete Command - Turn OFF

class LightOffCommand implements Command {

    private Light light;

    public LightOffCommand(Light light) {

        this.light = light;

    }

    public void execute() {

        light.turnOff();

    }

}

// Invoker Class

class RemoteControl {

    private Command command;

    public void setCommand(Command command) {

        this.command = command;

    }

    public void pressButton() {
```

```
        command.execute();
    }
}
// Main Class
public class Main {
    public static void main(String[] args) {
        // Receiver
        Light light = new Light();
        // Commands
        Command lightOn = new LightOnCommand(light);
        Command lightOff = new LightOffCommand(light);
        // Invoker
        RemoteControl remote = new RemoteControl();
        // Turn ON Light
        remote.setCommand(lightOn);
        remote.pressButton();
        // Turn OFF Light
        remote.setCommand(lightOff);
```

```
        remote.pressButton();

        System.out.println("\nCommand Pattern implemented successfully.");

    }

}
```

Output Screenshot:

Advantages

• Improves maintainability

• Encourages code reuse

• Enhances scalability

• Supports future enhancements

• Promotes clean software architecture

Real World Usage

The Command Pattern is commonly used in enterprise applications, web platforms, mobile systems, cloud services, and large-scale software solutions.

Exercise 10: MVC Pattern

Application Scenario: Student Management

Objective

The objective of this exercise is to understand and implement the MVC Pattern. The pattern helps developers create maintainable, reusable and scalable software systems.

Theory

MVC Pattern is an important object-oriented design approach. It addresses common software development challenges and promotes clean architecture. By separating responsibilities and reducing dependencies, the pattern improves code quality.

Algorithm

1. Identify participating classes.
2. Define interfaces or abstractions.
3. Implement concrete classes.
4. Apply the pattern structure.
5. Execute and validate the solution.

Sample Java Implementation

```
// Model Class

class Student {

   private String name;

   private int id;

   private String grade;
```

```
public Student(String name, int id, String grade) {
    this.name = name;
    this.id = id;
    this.grade = grade;
}
public String getName() {
    return name;
}
public int getId() {
    return id;
}
public String getGrade() {
    return grade;
}
public void setName(String name) {
    this.name = name;
}
public void setGrade(String grade) {
```

```
        this.grade = grade;
    }
}
// View Class
class StudentView {
    public void displayStudentDetails(String name, int id, String grade) {
        System.out.println("Student Details");
        System.out.println("---------------");
        System.out.println("Name  : " + name);
        System.out.println("ID    : " + id);
        System.out.println("Grade : " + grade);
    }
}
// Controller Class
class StudentController {
    private Student model;
    private StudentView view;
    public StudentController(Student model, StudentView view) {
```

```
    this.model = model;
    this.view = view;
}
public void setStudentName(String name) {
    model.setName(name);
}
public void setStudentGrade(String grade) {
    model.setGrade(grade);
}
public String getStudentName() {
    return model.getName();
}
public String getStudentGrade() {
    return model.getGrade();
}
public void updateView() {
    view.displayStudentDetails(
            model.getName(),
```

```
            model.getId(),
            model.getGrade());
    }
}
// Main Class
public class Main {
    public static void main(String[] args) {
        // Create Model
        Student student = new Student(
            "Rekha",
            101,
            "A");
        // Create View
        StudentView view = new StudentView();
        // Create Controller
        StudentController controller =
            new StudentController(student, view);
        // Display Initial Details
```

```
        System.out.println("Initial Student Details:");

        controller.updateView();

        // Update Student Information

        controller.setStudentName("Sakipalli Rekha");

        controller.setStudentGrade("A+");

        System.out.println("\nUpdated Student Details:");

        controller.updateView();

        System.out.println("\nMVC Pattern implemented successfully.");

    }

}
```

Output Screenshot:

Advantages

• Improves maintainability

• Encourages code reuse

• Enhances scalability

• Supports future enhancements

• Promotes clean software architecture

Real World Usage

The MVC Pattern is commonly used in enterprise applications, web platforms, mobile systems, cloud services, and large-scale software solutions.

Exercise 11: Dependency Injection

Application Scenario: Customer Service Management

Objective

The objective of this exercise is to understand and implement the Dependency Injection. The pattern helps developers create maintainable, reusable and scalable software systems.

Theory

Dependency Injection is an important object-oriented design approach. It addresses common software development challenges and promotes clean architecture. By separating responsibilities and reducing dependencies, the pattern improves code quality.

Algorithm

1. Identify participating classes.
2. Define interfaces or abstractions.
3. Implement concrete classes.
4. Apply the pattern structure.
5. Execute and validate the solution.

Sample Java Implementation

```
// Repository Interface

interface CustomerRepository {

   String findCustomerById(int id);

}
```

```
// Concrete Repository
class CustomerRepositoryImpl implements CustomerRepository {
    public String findCustomerById(int id) {
        return "Customer ID: " + id + ", Name: Rekha";
    }
}
// Service Class
class CustomerService {
    private CustomerRepository repository;
    // Constructor Injection
    public CustomerService(CustomerRepository repository) {
        this.repository = repository;
    }
    public void getCustomerDetails(int id) {
        System.out.println(repository.findCustomerById(id));
    }
}
// Main Class
```

```
public class Main {

   public static void main(String[] args) {

      // Injecting dependency through constructor

      CustomerRepository repository =

            new CustomerRepositoryImpl();

      CustomerService service =

            new CustomerService(repository);

      service.getCustomerDetails(101);

      System.out.println("\nDependency Injection implemented successfully.");

   }

}
```

Output Screenshot:

Advantages

• Improves maintainability

• Encourages code reuse

• Enhances scalability

• Supports future enhancements

• Promotes clean software architecture

Real World Usage

The Dependency Injection is commonly used in enterprise applications, web platforms, mobile systems, cloud services, and large-scale software solutions.

Conclusion

The successful completion of all eleven exercises demonstrates a strong understanding of software design principles and object-oriented development practices. These patterns provide proven solutions to recurring design problems and are widely adopted in modern software engineering.